30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 18

Performance — Part 2: App Size, Battery, Baseline Profiles & Macrobenchmark



Esmaeeil Moradi
@esmaeeil-moradi700

Why this part matters in 2026

Part 1 was frames and memory. Part 2 is everything around them: the bytes users download, the battery and data you burn, how fast your code runs on day one — and how Google judges you in the field.

- R8 full mode is the default since AGP 8.0 — and it changes what keep rules must say
- Google Play builds split APKs from your App Bundle; nobody downloads the file you upload
- Battery is a quality signal: Android vitals reports excessive wake locks next to crashes and ANRs
- Baseline and Startup Profiles fix first-run slowness that the JIT can't
- Macrobenchmark proves a change in the lab; Android vitals confirms it in production

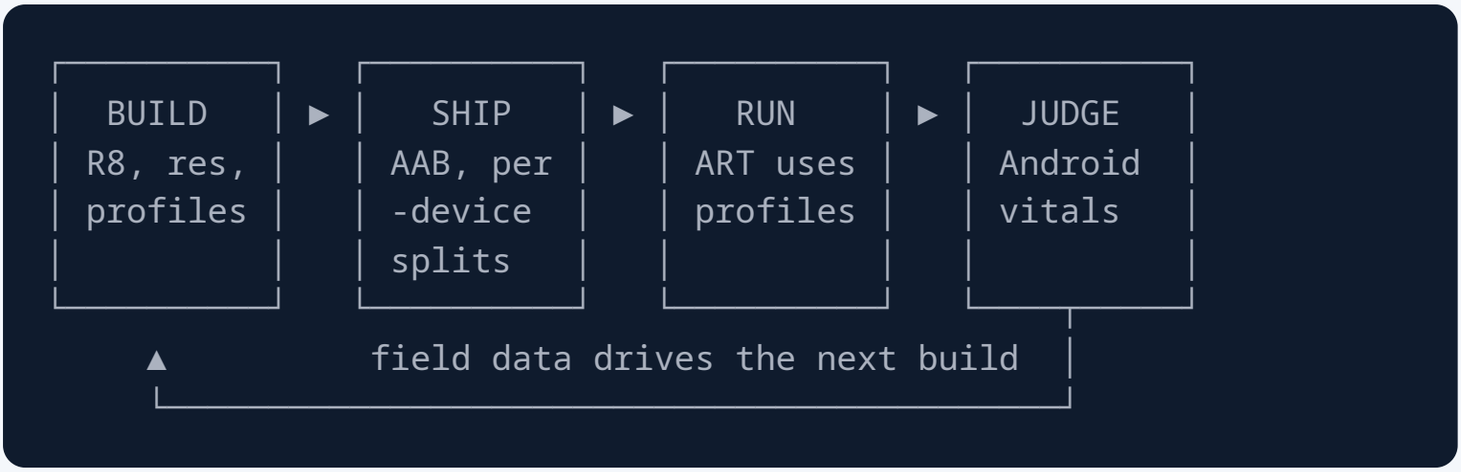
Level	Questions
Junior	What makes an app big · App Bundles · What drains battery
Mid	R8 full mode · Size levers · Network efficiency · How ART runs your code
Senior	Baseline & Startup Profiles · Macrobenchmark in depth · Android vitals
Architect	Size as architecture · Efficient sync · Lab + field toolchain



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18

Mental model: build → ship → run → judge



- **Build:** R8 shrinks code and resources; Baseline and Startup Profiles are packaged in
- **Ship and run:** Play delivers only what a device needs; ART compiles the hot paths from the profile
- **Judge:** vitals from real users drive the next release; Macrobenchmark guards every step in the lab

NAME THE TRADE-OFF

Size, battery and speed pull against each other: full AOT compilation is fast but big; prefetching is fast but costs data. A senior answer says what it trades.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18



Junior

What makes an app big · App Bundles · What drains battery



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What makes an app big?

Inside the APK	What it is	Main lever
`classes*.dex`	Your code plus every library	R8
`res/`, `resources.arsc`	Layouts, images, strings	Resource shrinking, vectors, WebP
`lib/<abi>/`	Native `.so` libraries	Only the ABIs a device needs
`assets/`	Raw files: ML models, fonts, media	Play Asset Delivery, download later

- **APK Analyzer** (Build → Analyze APK): raw vs download size, compare two builds, look inside the dex files
- **Download size** (compressed, what the user waits for) ≠ **size on device** (grows with compiled code and data)

INTERVIEW TRAP

Measuring the size of a **debug** APK. Debug builds aren't shrunk or optimized — only the release build tells you anything.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. App Bundles: let Play split for you

```
app-release.aab ──▶ Google Play ──▶ this device installs:
 (upload key)       (signs with      base.apk
                     app signing     split_config.arm64_v8a.apk
                     key)            split_config.xxhdpi.apk
                                     split_config.fa.apk
```

- Required for new apps on Google Play since August 2021
- **Play App Signing:** you sign uploads with an *upload key*; Play signs the APKs it generates

```
# Per-device sizes from the bundle you upload
bundletool build-apks --bundle=app-release.aab \
  --output=app.apks
bundletool get-size total --apks=app.apks
```

INTERVIEW TRAP

"My `.aab` is 40 MB, so users download 40 MB." — Nobody downloads the bundle. Play generates split APKs per device — one ABI, one density, the user's languages — compressed. Measure with bundletool or Play Console's app bundle explorer.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Two bundle traps

INTERVIEW TRAP

"Configure Gradle APK splits per ABI and density to cut size." — On Play, the bundle already does this per device. Gradle `splits {}` only matters when you distribute APKs yourself (other stores, direct download).

INTERVIEW TRAP

"Language splits are free." — With an in-app language picker, languages the device doesn't use aren't installed. Request them at runtime with `SplitInstallManager`, or turn language splitting off:

```
android {
    bundle {
        language { enableSplit = false }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What drains battery

Drain	Why	Better
Partial wake locks	The CPU can't sleep	WorkManager (holds the lock for you) or a short, timed lock
Frequent network	Each transfer wakes the radio, which stays in a high-power state for a while afterwards	Batch, prefetch, push instead of poll
Location	GPS is expensive	Lowest priority that works; batched updates
Wake-ups	Alarms and jobs that wake the device	Inexact alarms, WorkManager constraints (Days 15–16)
Background CPU	Work nobody sees	Stop it with the lifecycle

Tools: Android Studio's **Power Profiler** (measured power rails on supported devices, e.g. Pixel 6 and newer), `adb shell dumpsys batterystats`, Android vitals.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. If you need a wake lock, time it

```
val pm = context.getSystemService(PowerManager::class.java)
val lock = pm.newWakeLock(
    PowerManager.PARTIAL_WAKE_LOCK, "myapp:upload",
)
lock.acquire(10 * 60 * 1000L) // always with a timeout
try {
    uploadPendingPhotos()
} finally {
    if (lock.isHeld) lock.release()
}
```

- Better still: no wake lock of your own — WorkManager keeps the device awake while `doWork()` runs
- Tag it `app:purpose` so it's identifiable in `batterystats` and vitals

INTERVIEW TRAP

"Doze makes wake-lock bugs harmless." — Deep Doze starts only after the device has been unplugged, still and screen-off for a while. Until then your lock really keeps the CPU awake — and Android vitals counts it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18

Mid-Level

R8 full mode · Size levers · Network efficiency · How ART runs your code



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. R8: shrink, optimize, obfuscate

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true    // R8 on code
            isShrinkResources = true  // then unused resources
            proguardFiles(
                getDefaultProguardFile(
                    "proguard-android-optimize.txt"
                ),
                "proguard-rules.pro",
            )
        }
    }
}
```

- **Shrinks** unused code, **optimizes** (inlining, class merging, enum unboxing), **obfuscates** names; then unused resources go
- **Full mode is the default since AGP 8.0** — it assumes less than ProGuard-compatible mode did

INTERVIEW TRAP

"Any default ProGuard file will do." — `proguard-android.txt` contains `-dontoptimize`: shrinking without R8's biggest wins. And R8 isn't "just obfuscation" — shrinking and optimization make the app smaller **and** faster (security: Day 20).



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Full mode and keep rules

Full mode doesn't keep what reflection *might* need: keeping a class doesn't keep its default constructor, and generic `Signature` attributes survive only on kept items. Gson models and classes loaded by name need explicit rules.

```
# Narrow and commented. Never: -keep class com.example.** { *; }
-keep class * implements com.example.Plugin {
    <init>();
}
# Why is this still in the APK?
-whyareyoukeeping class com.example.feed.FeedMapper
```

- Libraries ship their own consumer rules; prefer libraries that don't need reflection
- Dynamic lookups (`getIdentifier()`) make resource shrinking conservative; `res/raw/keep.xml` with `tools:keep` / `tools:discard` controls it

INTERVIEW TRAP

"Full mode broke the release, so turn it off." — `android.enableR8.fullMode=false` hides the problem. Find the reflective code (crash stack + `mapping.txt`), add a narrow rule, and test that path on a release build.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Size levers beyond R8

Lever	What to do
Images	Vector drawables for icons; WebP for photos (Studio can convert)
Dependencies	Check the release-build size delta of every new SDK — one SDK can outweigh a feature
Reflection	`kotlin-reflect` and reflective JSON libraries are heavy and hard to shrink
Native code	Only needed ABIs (the bundle splits them); strip debug symbols
Big assets	ML models, video: Play Asset Delivery or download on demand

ANDROID 15+ · 16 KB PAGES

Google Play requires apps with native code that target Android 15+ to support **16 KB memory pages**. Rebuild your `.so` files with a current NDK and AGP (both align for 16 KB by default in recent versions) and get updated builds from SDK vendors.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Two size myths

INTERVIEW TRAP

"Use Gson or Moshi — they're lightweight." — Gson is reflection-based: slower, and fragile under R8 full mode. Prefer compile-time serializers: kotlinx.serialization, or Moshi with codegen.

INTERVIEW TRAP

"Replace enums with `Int` *constants to save space."* — Decade-old advice. R8 unboxes simple enums. Write readable code and measure the release build.

FOLLOW-UP

"How do you find what grew?" — Compare the old and new release APKs in APK Analyzer: it diffs dex, resources and native libraries file by file.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Network efficiency: fewer wake-ups, fewer bytes

- **Batch** requests and **prefetch** what the user likely needs while the radio is already on
- **Don't poll** — push a signal with FCM and sync once (Day 16)
- **Send less:** pagination, field projections, images at display size from the CDN
- **Compress and cache:** OkHttp handles gzip transparently; HTTP caching is Day 11
- **Respect the network:** metered connections and Data Saver
- Inspect real traffic with Android Studio's **Network Inspector**

INTERVIEW TRAP

"Polling every minute is fine — the payload is tiny." — The cost is the radio wake-up and the high-power tail after it, not the bytes. Sixty tiny requests an hour are sixty radio wake-ups.

INTERVIEW TRAP

"Add a gzip interceptor to OkHttp." — OkHttp already asks for gzip and decompresses it — unless you set `Accept-Encoding` yourself; then decompression is your job.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Adapt to the user's network

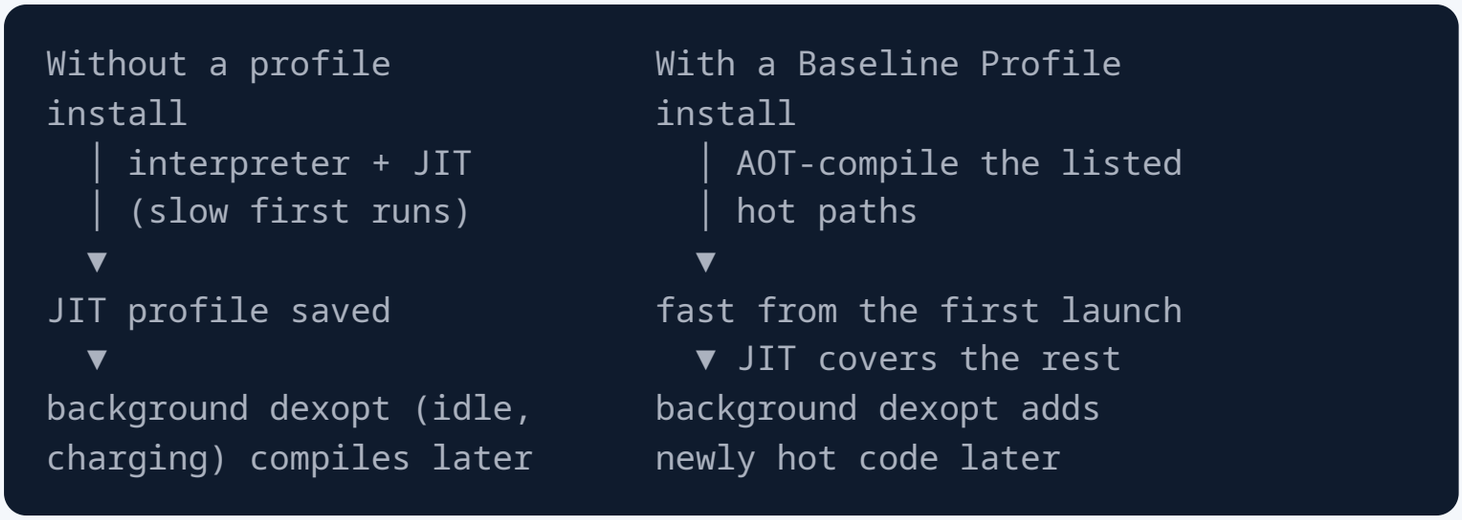
```
fun prefetchPolicy(cm: ConnectivityManager): Prefetch {
    val caps = cm.getNetworkCapabilities(cm.activeNetwork)
        ?: return Prefetch.None
    val dataSaver = cm.restrictBackgroundStatus ==
        RESTRICT_BACKGROUND_STATUS_ENABLED
    return when {
        dataSaver -> Prefetch.None
        caps.hasCapability(NET_CAPABILITY_NOT_METERED) ->
            Prefetch.Full
        else -> Prefetch.ThumbnailsOnly
    }
}
```

- Heavy background transfers: WorkManager with `NetworkType.UNMETERED`, maybe `setRequiresCharging(true)`
- Watch changes with `registerDefaultNetworkCallback` instead of checking once at startup



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. How ART runs your code



- **Cloud Profiles:** Play aggregates profiles from users of a version and ships them with later installs
- **Baseline Profile:** you ship the hot-path list inside the app; it's used at install time. For non-Play installs, `profileinstaller` hands it to ART for the next background dexopt
- Rule format: `HSPLcom/example/FeedViewModel;->load()V` — **H**ot, **S**tartup, **P**ost-startup



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Profile traps

INTERVIEW TRAP

"Cloud Profiles make Baseline Profiles unnecessary." — Cloud Profiles take time to build up for every new version, and installs outside Play don't get them. A Baseline Profile works from the first install.

INTERVIEW TRAP

"Why not AOT-compile everything?" — Full compilation makes the app bigger on disk and installs and updates slower, and most code is cold. Profile-guided compilation targets what users actually run.

INTERVIEW TRAP

"I tested my Baseline Profile in a debug build." — Debuggable apps don't use profile-guided compilation. Measure a non-debuggable release build.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18



Senior

Baseline & Startup Profiles · Macrobenchmark in depth · Android vitals



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. The profile pipeline: Gradle

```
// :app — the consumer
plugins {
    id("com.android.application")
    id("androidx.baselineprofile")
}
dependencies {
    implementation(libs.androidx.profileinstaller)
    baselineProfile(project(":baselineprofile"))
}

// :baselineprofile — the producer (com.android.test)
android { targetProjectPath = ":app" }
baselineProfile {
    managedDevices += "pixel6Api34"
    useConnectedDevices = false
}
```

- `./gradlew :app:generateBaselineProfile` runs the generator (here on a Gradle Managed Device) and writes the profiles into the app's source set — commit them
- AndroidX libraries such as Compose ship their own profiles; they're merged with yours



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. The generator: record real journeys

```
@RunWith(AndroidJUnit4::class)
class BaselineProfileGenerator {
    @get:Rule val rule = BaselineProfileRule()

    @Test fun generate() = rule.collect(
        packageName = "com.example.app",
        includeInStartupProfile = true,
    ) {
        pressHome()
        startActivityAndWait()
        // the critical journeys, not just startup
        device.findObject(By.res("feed")).fling(Direction.DOWN)
        device.findObject(By.text("Profile")).click()
    }
}
```

- **Baseline Profile** → AOT compilation on the device. **Startup Profile** → used by R8 at **build time** to put startup classes in the primary DEX file (DEX layout); needs R8 enabled
- Compose: set `testTagsAsResourceId = true` in a root `Modifier.semantics` so `By.res()` finds test tags

INTERVIEW TRAP

"We generated it once." — Profiles describe code paths, and code changes every sprint. Regenerate for each release and verify with a benchmark.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Prove the profile works

```
@Test fun startupNoCompilation() =
    startup(CompilationMode.None())

@Test fun startupBaselineProfile() = startup(
    CompilationMode.Partial(BaselineProfileMode.Require)
)

private fun startup(mode: CompilationMode) =
    rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(StartupTimingMetric()),
        compilationMode = mode,
        startupMode = StartupMode.COLD,
        iterations = 10,
    ) {
        pressHome()
        startActivityAndWait()
    }
```

- `BaselineProfileMode.Require` fails the run if no profile is installed — no silent "success"
- In the field, `ProfileVerifier.getCompilationStatusAsync()` from `profileinstaller` reports whether this install was compiled with the profile



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Macrobenchmark: set up the target

```
// app/build.gradle.kts — the baselineprofile plugin
// creates an equivalent "benchmarkRelease" for you
buildTypes {
    create("benchmark") {
        initWith(getByName("release"))
        signingConfig = signingConfigs.getByName("debug")
        matchingFallbacks += listOf("release")
        isDebuggable = false
    }
}
```

```
<application>
    <profileable android:shell="true" tools:targetApi="29" />
</application>
```

- A separate `com.android.test` module drives the installed app from another process through UiAutomator — like a user
- The target must be **non-debuggable**, with R8 on, and **profileable** so traces can be captured



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. A scroll benchmark

```
@Test fun scrollFeed() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(FrameTimingMetric()),
    compilationMode = CompilationMode.Partial(),
    startupMode = StartupMode.WARM,
    iterations = 10,
    setupBlock = { startActivityAndWait() },
) {
    val feed = device.findObject(By.res("feed"))
    // keep flings away from the gesture-navigation edges
    feed.setGestureMargin(device.displayWidth / 5)
    feed.fling(Direction.DOWN)
    device.waitForIdle()
}
```

- `setupBlock` isn't measured: navigate there first. The trailing lambda is the measured journey
- Results: P50, P90, P95 and P99 per metric, plus a Perfetto trace per iteration you can open from Android Studio



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Metrics and traps

Metric	Measures
`StartupTimingMetric`	`timeToInitialDisplayMs`, `timeToFullDisplayMs`
`FrameTimingMetric`	`frameDurationCpuMs`, `frameOverrunMs` (API 31+)
`TraceSectionMetric`	Your own `trace("…")` sections
`MemoryUsageMetric`, `PowerMetric`	Experimental; power needs a device with power rails

INTERVIEW TRAP

"Report the average FPS." — Averages hide jank. Compare **frame duration and overrun percentiles** (P90, P99), and TTID/TTFD for startup.

INTERVIEW TRAP

"A stable emulator is fine for performance tests." — Emulator timings don't reflect real hardware, and Macrobenchmark reports an error on emulators by default. Use physical devices.

FOLLOW-UP

"Macrobenchmark or Microbenchmark?" — Macro: user journeys (startup, scroll, navigation) from outside the app. Micro: one hot function in a loop, in-process — a parser, a diff algorithm.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Android vitals: how Google judges you

Core vital	Bad-behavior threshold
User-perceived crash rate	1.09% overall
User-perceived ANR rate	0.47% overall
Either one, per phone model	8%

- More vitals: slow cold, warm and hot starts (Day 1), slow rendering, frozen frames (> 700 ms), excessive partial wake locks
- Above an overall threshold, Play may make the app less visible; above the per-model threshold, users of that model may see a warning on the store listing
- Data comes from users who opted in to share usage and diagnostics; also available through the **Play Developer Reporting API**



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Reading vitals correctly

INTERVIEW TRAP

"Crash rate = crashes ÷ sessions." — It's the share of **daily active users** who hit at least one user-perceived crash. One user crashing 50 times counts once; a rare crash spread over many users counts a lot.

INTERVIEW TRAP

"Our overall rate is fine, so we're fine." — The 8% threshold is checked per phone model. One popular low-end model above it can get a warning on its store listing.

FOLLOW-UP

"How do you get from an ANR cluster to a fix?" — Play Console groups ANRs with main-thread stacks. Add `ApplicationExitInfo` traces (Day 1), then reproduce with a system trace.

BATTERY COUNTS TOO

Treat the excessive partial wake lock metric like crash and ANR rates: alert on it per release, not after users complain.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18

Architect

Size as architecture · Efficient sync · Lab + field toolchain



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. App size as an architecture concern

Size grows through **dependencies and assets**, rarely through your own code — so govern those:

- Every new SDK comes with a release-build size diff in the PR (budgets in CI: Day 28); one library per job

Delivery option	Use for	Cost
Install-time modules	Modularization only	No size win
On-demand feature modules	Large, rarely used features	Runtime download, install states, testing
Conditional delivery	Some devices or countries	A complex release matrix
Asset packs: install-time, fast-follow, on-demand	Big models, media, game levels	Asset lifecycle to manage

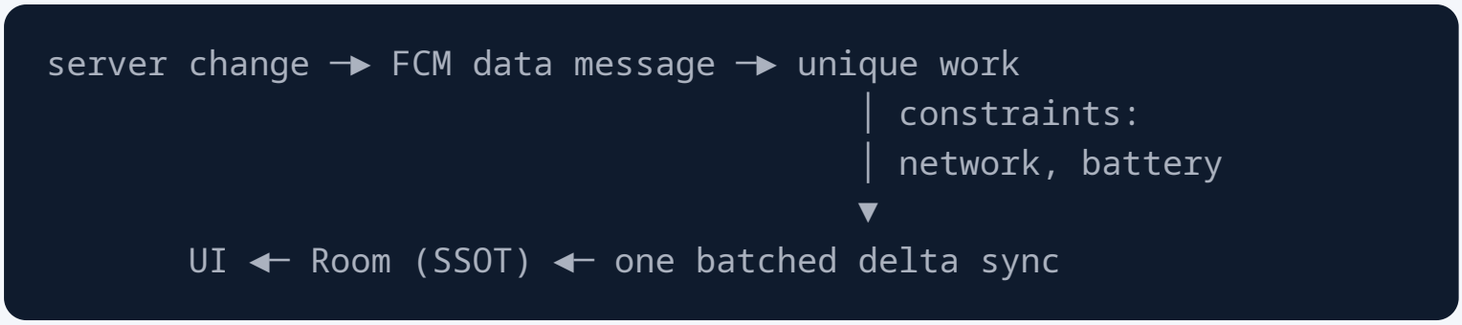
INTERVIEW TRAP

"Move features to on-demand modules to shrink the app." — Users then wait for a download exactly when they need the feature, and you own install failures and testing. Worth it only for big, rarely used features (Day 25).



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Battery- and network-friendly sync



- **Push, don't poll:** the server says "something changed"; the app syncs once (Day 16)
- **Coalesce:** unique work turns ten signals into one sync
- **Deltas** (`since=<cursor>`) instead of full downloads; exponential backoff on failure
- **Defer the heavy part** — media upload, prefetch — to unmetered networks or charging



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The sync request

```
fun onSyncSignal(workManager: WorkManager) {
    val request = OneTimeWorkRequestBuilder<SyncWorker>()
        .setConstraints(
            Constraints.Builder()
                .setRequiredNetworkType(NetworkType.CONNECTED)
                .setRequiresBatteryNotLow(true)
                .build()
        )
        .setBackoffCriteria(
            BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS,
        )
        .build()
    workManager.enqueueUniqueWork(
        "sync", ExistingWorkPolicy.KEEP, request,
    )
}
```

INTERVIEW TRAP

`KEEP` drops a signal that arrives while a sync is **running**. Re-check the cursor at the end of the sync, or use `APPEND_OR_REPLACE`.

INTERVIEW TRAP

"A 15-minute periodic job is the safe default." — That's still polling: every run wakes the device and the radio. Push-triggered sync first; periodic work only as a slow safety net.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The lab + field toolchain

Question	Lab, before release	Field, after release
Fast startup?	`StartupTimingMetric`	Vitals slow starts
Smooth scrolling?	`FrameTimingMetric` + Perfetto	JankStats, slow and frozen frames
Profile applied?	`CompilationMode` comparison	`ProfileVerifier`
Battery?	Power Profiler, `PowerMetric`	Vitals wake locks
Why?	A trace per iteration	`ProfilingManager` (Android 15+), `ApplicationExitInfo`

- The lab is repeatable and catches regressions early; the field shows the device mix, data sizes and thermal conditions you never tested



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Closing the loop every release

1. Regenerate Baseline and Startup Profiles; run Macrobenchmarks on physical devices
2. Staged rollout: watch vitals per version and per phone model (Reporting API → alerts)
3. Regression? Pull field traces and ANR clusters, reproduce in the lab, fix, add a benchmark

Budgets, dashboards and performance culture: Day 28.

INTERVIEW TRAP

"We have field monitoring, so we don't need benchmarks." — Field data arrives after users suffer, and it's noisy. Lab benchmarks stop the regression before release. You need both.

SENIOR ANSWER IN ONE LINE

"Ship fewer bytes, wake the device less, compile the hot path ahead of time — and prove every claim with a benchmark in the lab and vitals in the field."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 18 · ARCHITECT

Day 18 — cheat sheet

- Nobody downloads your `.aab`: Play builds per-device split APKs. Measure release builds with bundletool and APK Analyzer.
- R8 full mode is the default since AGP 8.0: shrink, optimize, obfuscate. Narrow keep rules; `proguard-android-optimize.txt`.
- `isShrinkResources` removes resources no remaining code uses; `keep.xml` for dynamic lookups.
- In-app language picker + bundle → request languages at runtime or disable language splits.
- Native code targeting Android 15+ must support 16 KB pages on Google Play.
- Battery: timed wake locks (or none — WorkManager), batched network, push over poll.
- Baseline Profile → AOT at install. Startup Profile → DEX layout at build time. Regenerate every release.
- Macrobenchmark: non-debuggable, profileable, physical device; compare `CompilationMode`s; percentiles, not averages.
- Vitals: crashes 1.09%, ANRs 0.47% of daily active users; 8% per phone model.
- The lab prevents regressions; the field proves the fix.

Day 19 is next → Security, part 1: threat modeling, secure storage & permissions. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700